Parallel NMS-SIFT for Content-Based Image Retrieval

Why more threads didn't help, and what did

Musa Waghu | Senior research project in parallel processing

The project in four numbers

1.4%

average gain from 16 threaded tiles over 4 in 2023: not significant

1.0

CPU cores actually busy with threads, on an 8-core CPU

6.65×

speedup with processes at 16 tiles, the fix

−7.9%

keypoints lost at tile borders with 16 tiles

Part 1

Background: image retrieval and SIFT

Why content-based image retrieval

Text-based (TBIR)

- Experts tag images by hand
- Slow to annotate
- Tags are subjective and inconsistent

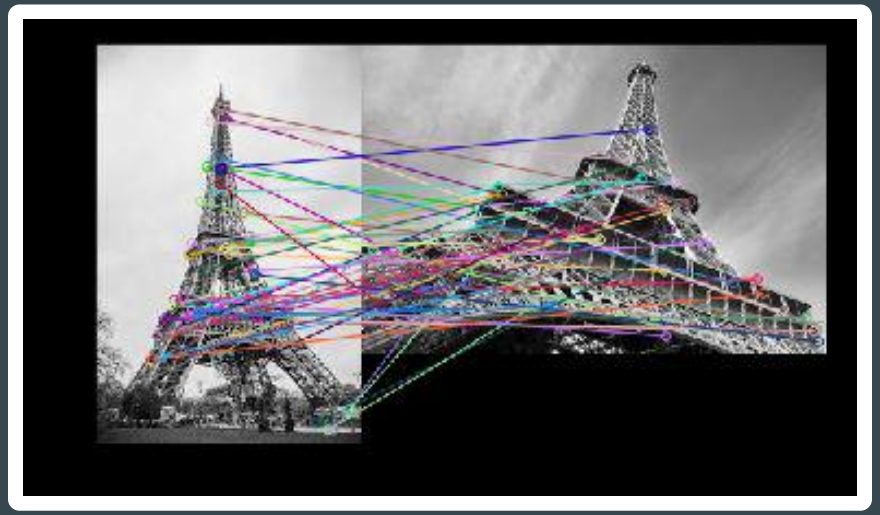
Content-based (CBIR)

- Query with an image, compare visual features
- No manual labelling
- Cost: feature extraction for every image

97 zettabytes of data were created in 2022 [1]. Feature extraction speed decides whether CBIR scales.

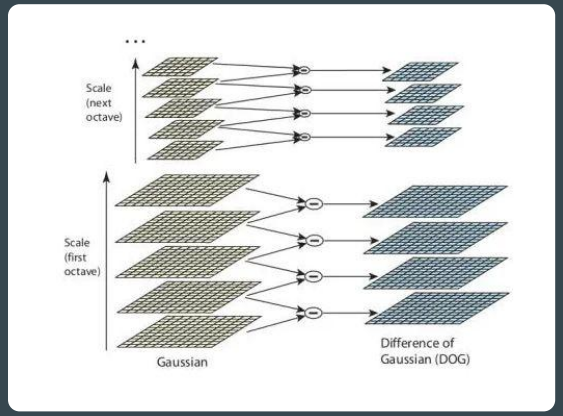
SIFT: keypoints that survive scale and rotation

1. **Scale-space extrema**
   Gaussian pyramid, Difference of Gaussians
2. **Keypoint localisation**
   Reject low contrast and edges
3. **Orientation**
   Dominant gradient direction
4. **Descriptor**
   128-d gradient histogram

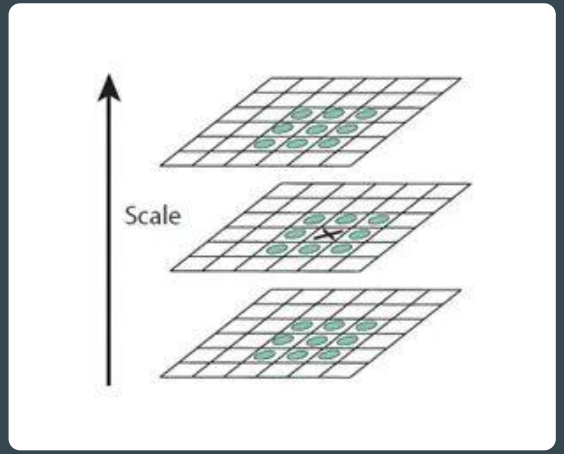


SIFT matches the same landmark across scale and viewpoint (Lowe, 1999 [4])

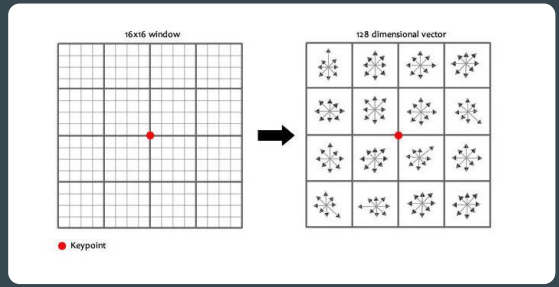
Inside SIFT



Difference of Gaussians across scales



Each pixel checked against 26 neighbours



16×16 window becomes a 128-d descriptor

NMS–SIFT: keep the strongest keypoint nearby

- Sort keypoints by response strength
- Drop any keypoint within 10 px of a stronger one
- Fewer, more distinctive keypoints for matching



NMS in object detection. Image credit: learnopencv.com

Non-maximum suppression is borrowed from object detection, where it removes overlapping boxes [5].

Part 2

The 2023 study

Hypothesis: more tiles, more threads, faster



4 tiles, 4 threads



16 tiles, 16 threads

Hypothesis

SIFT's work is local, so each tile can be processed on its own thread. Splitting into 16 tiles should be faster than 4.

Experimental setup

Dataset

Oxford 102 Category Flowers [6]: the first 50 images

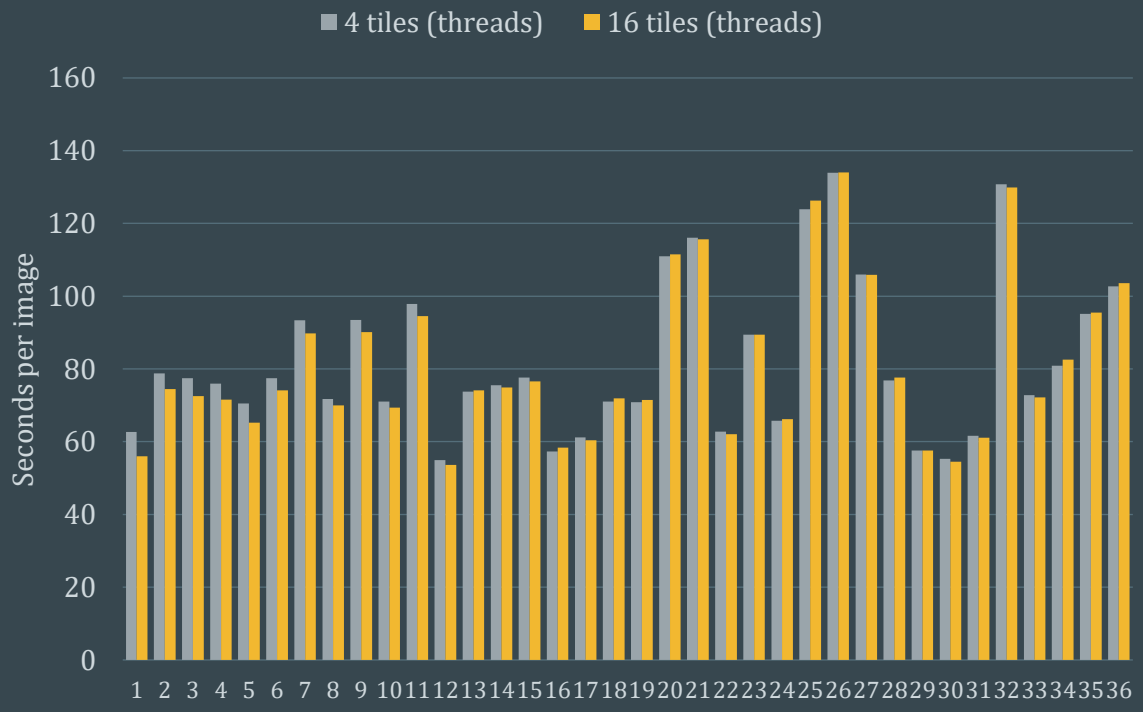
Hardware

AMD Ryzen 7 3700X, 8 cores / 16 threads. The 2026 rerun used the same CPU model.

Code

Pure-Python NMS-SIFT by S. Jadhav and Dr. J. Korah, with tiling and threading added

2023 result: no significant improvement



24 / 36

images faster with 16 tiles, the rest slower

+1.4%

average gain: not significant

—5%

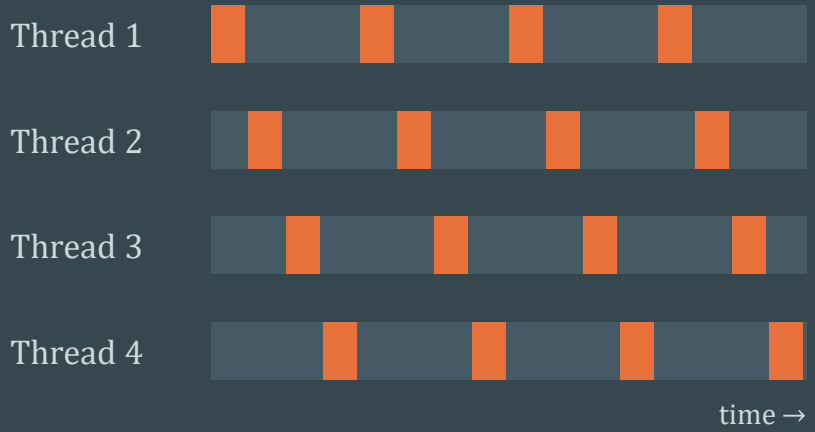
keypoints with 16 tiles vs 4

Part 3

Diagnosis: why threads didn't help

The culprit: Python's Global Interpreter Lock

4 threads (GIL)



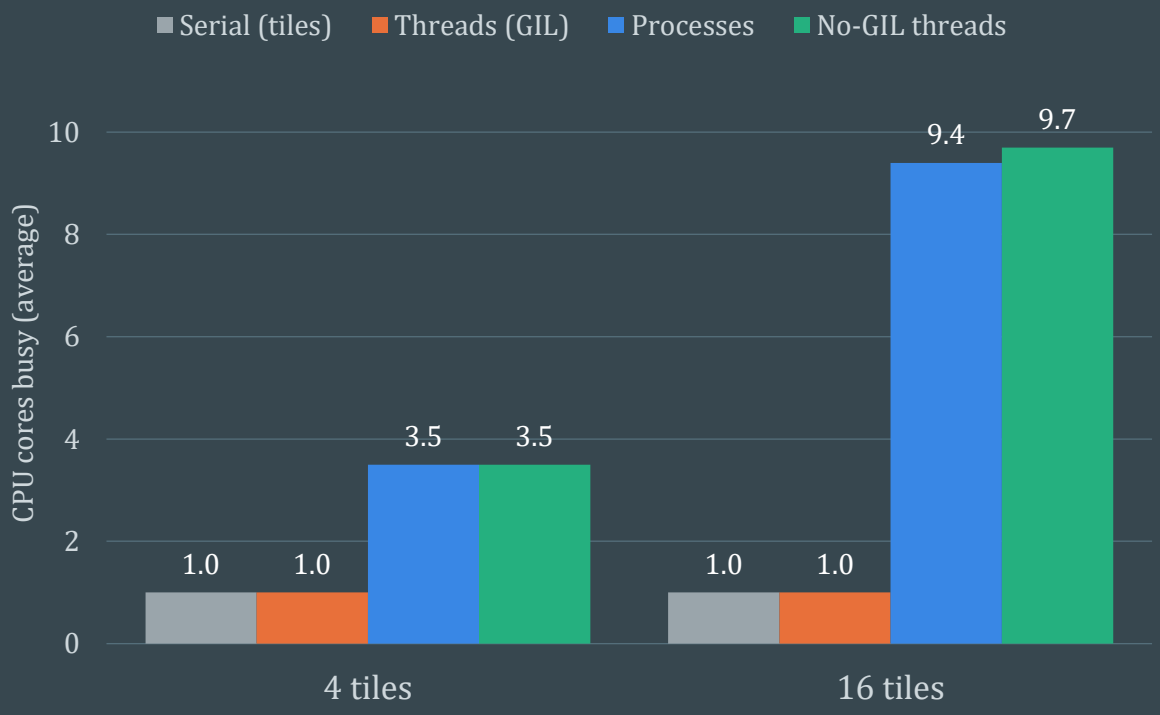
Only one thread runs Python at a time. They take turns on one core.

4 processes



Each process has its own interpreter and its own GIL, so all four run at once.

Measured: threads kept exactly one core busy



How it's measured

Each worker records its own CPU time. Total CPU time ÷ wall-clock time = cores busy. Above 8 means SMT siblings are in use.

Part 4

The fix: processes, and Python without the GIL

Rerun: same images, seven configurations

Configuration	What it tests
Serial, whole image	Baseline
Serial, 4 / 16 tiles	New control: tiling with no parallelism
Threads, 4 / 16 tiles	The 2023 method
Processes, 4 / 16 tiles	The fix: one interpreter per worker
No-GIL threads (3.14t)	Python's new free-threaded build

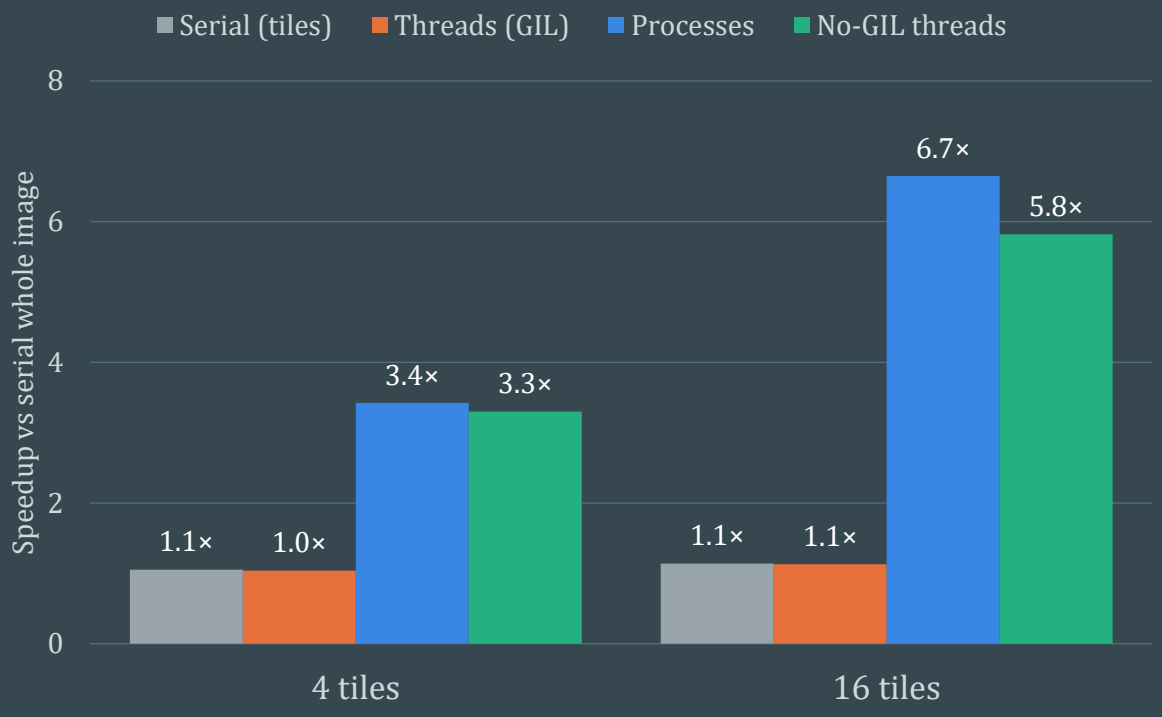
Same images as 2023?

36 / 36 16-tile keypoint counts match the slides exactly

44 / 50 4-tile counts match; the rest are within 2 keypoints

Every parallel run must give keypoints identical to serial on the same tiles.

Processes: 6.65× faster at 16 tiles



Threads (GIL)

1.04× / 1.13×: only the tiling effect

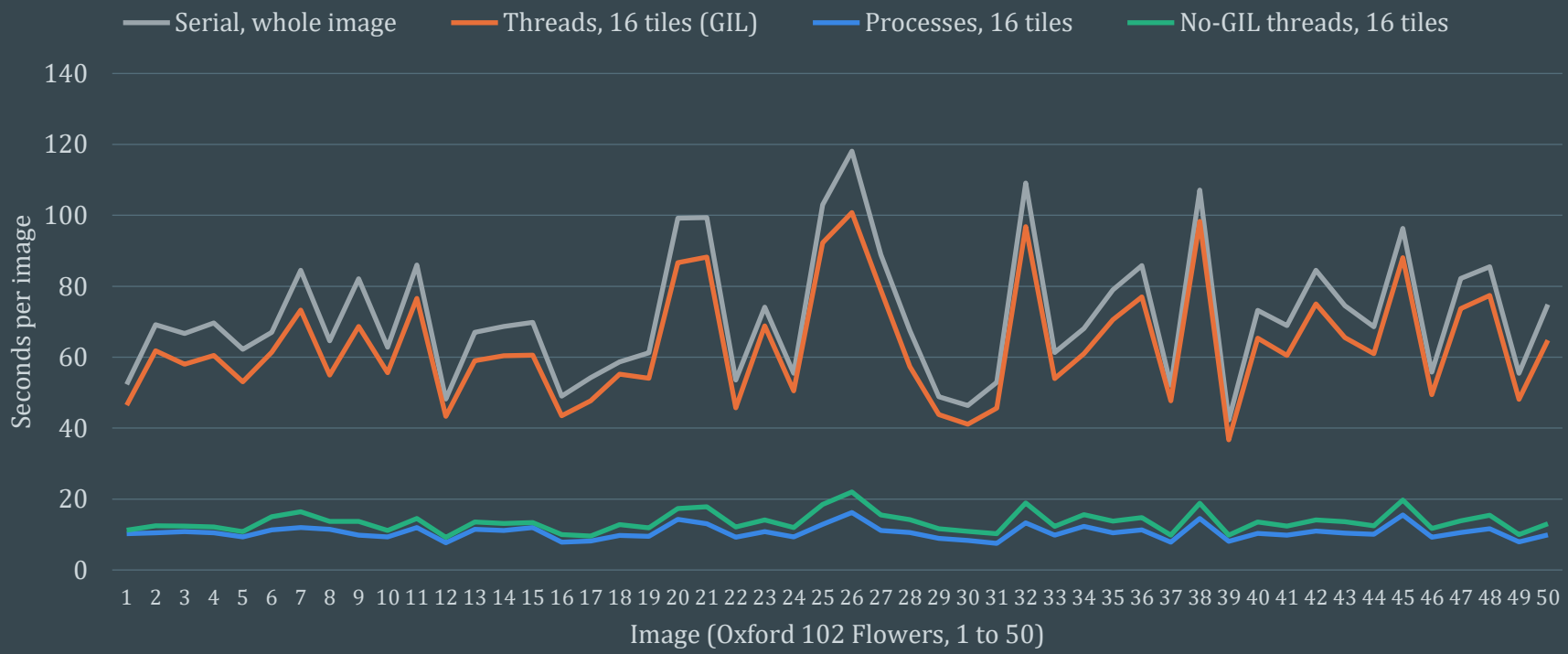
Processes

3.42× / 6.65×

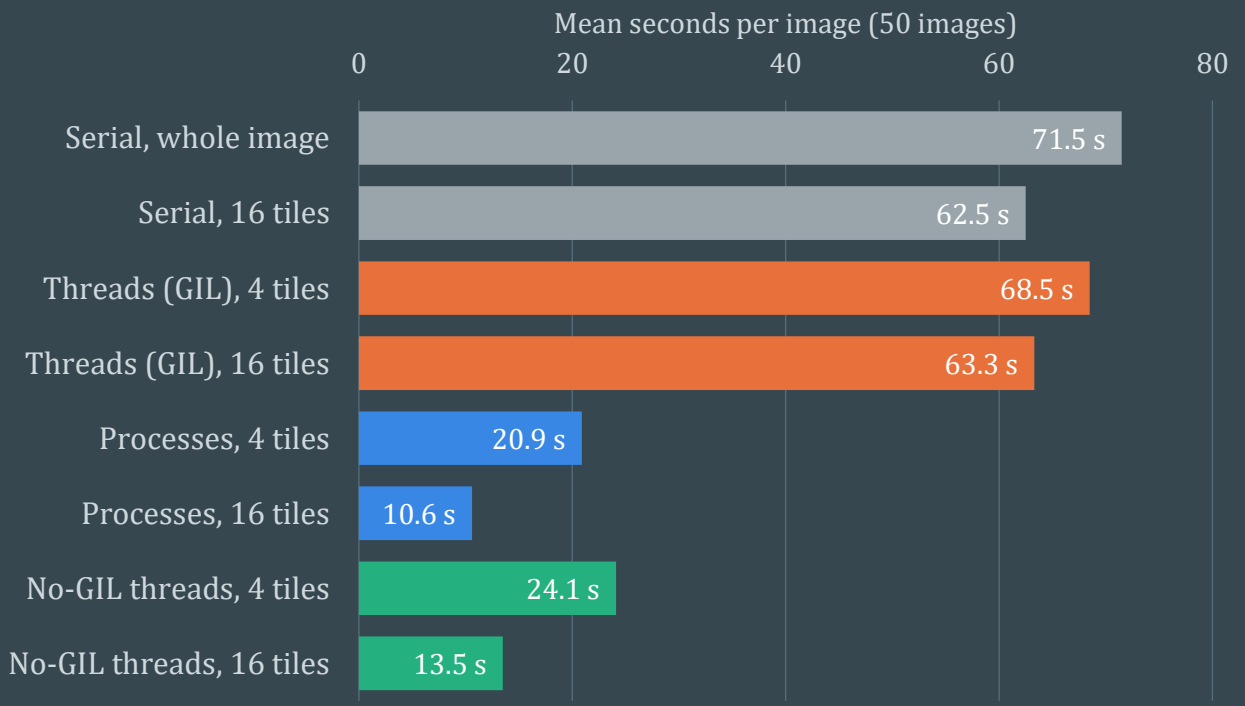
16 beats 4

With real parallelism, the 2023 hypothesis holds

Every image, same story



Processing time: seconds per image



59.6 min

serial, all 50 images

8.8 min

16 processes, same 50 images

−85%

time per image: 71.5 s to 10.6 s

Processing time per version

Version	Mean per image	Fastest / slowest	All 50 images
Serial, whole image	71.5 s	42.3 / 118.1 s	59.6 min
Serial, 4 tiles	67.9 s	39.6 / 107.2 s	56.6 min
Serial, 16 tiles	62.5 s	36.1 / 98.3 s	52.1 min
Threads (GIL), 4 tiles	68.5 s	40.2 / 107.9 s	57.1 min
Threads (GIL), 16 tiles	63.3 s	36.7 / 100.8 s	52.7 min
Processes, 4 tiles	**20.9 s**	13.1 / 34.6 s	**17.5 min**
Processes, 16 tiles	**10.6 s**	7.5 / 16.2 s	**8.8 min**
No-GIL: serial, whole image	79.5 s	47.8 / 127.9 s	66.2 min
No-GIL: threads, 4 tiles	24.1 s	15.0 / 39.2 s	20.1 min
No-GIL: threads, 16 tiles	13.5 s	9.2 / 22.0 s	11.3 min

Wall-clock time to extract keypoints and descriptors. Other rows: Python 3.14 with the GIL. No-GIL: free-threaded 3.14t.

Python 3.14 without the GIL: threads finally scale

5.82×

No-GIL threads at 16 tiles (3.30× at 4), close to processes

+11%

Slower single-threaded: the cost of a thread-safe interpreter

0

OpenCV builds for free-threaded Python. I reimplemented 4 calls in NumPy

PEP 703 made the GIL optional [7]. For this workload it already works, though libraries are still catching up.

The hidden cost: keypoints lost at tile borders

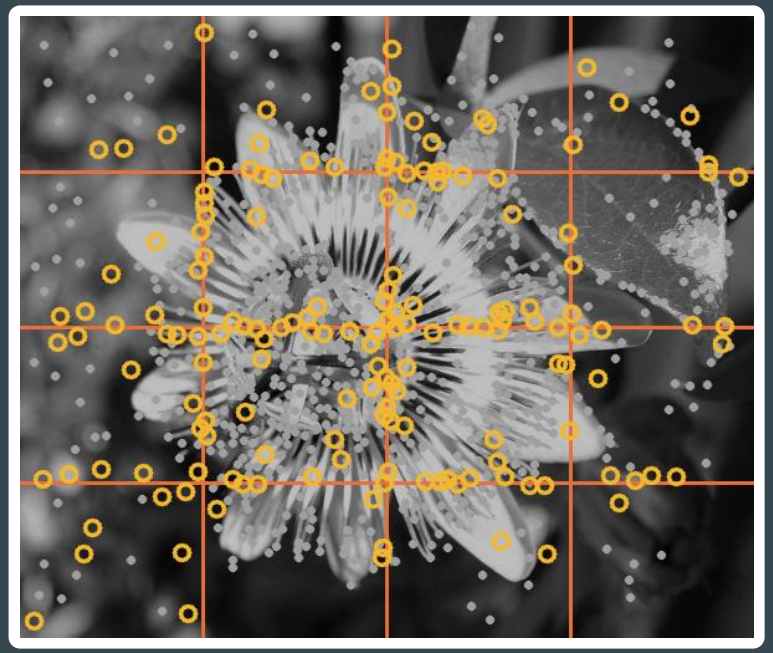


Image 1, 16 tiles. Gold rings: whole-image keypoints with no 16-tile match nearby. They cluster along the cuts.

Tiling	Keypoints	Lost
Whole image	1,290	n/a
4 tiles	1,251	−3.1%
16 tiles	1,191	**−7.9%**

- Each tile skips a 5-pixel border
- Descriptor windows are clipped at the cut
- Worst image loses 12.1% at 16 tiles

Bugs found while rebuilding



Ordering race

Threads saved results as they finished (e.g. 3, 1, 0, 2), but merging assumed order 0 to 3. Keypoints could land in the wrong quadrant.



Pickling error

cv2.KeyPoint can't be pickled, which blocks saving results and multiprocessing. Fixed by passing plain tuples.



Matching crash

The retrieval step passed the wrong image shape to perspectiveTransform.

Part 5

Conclusions and future work

Conclusions

1

The GIL, not the idea, sank 2023

Pure-Python loops on threads never used more than one core.

2

The small 2023 gain came from tiling

Fewer keypoints meant less work, at the cost of accuracy.

3

Real parallelism works

Processes: 6.65×. No-GIL threads: 5.82×. 16 tiles beat 4.

Future work

Get further around the GIL

Compile hot loops with Numba or Cython so they release the GIL; vectorise with NumPy; scale across machines

Move SIFT to a GPU

Per-pixel and per-keypoint work is ideal for GPUs (e.g. CudaSift, PopSift). Batch images to hide transfer costs

Stop losing keypoints

Overlapping tiles plus NMS across seams, or detect on the whole image and parallelise only descriptors

Measure retrieval quality

Rerun FLANN + RANSAC matching to see whether lost keypoints hurt accuracy

Perspective: compiled C++ SIFT is about 600× faster (88 ms vs 52 s). Faster cores first, then more cores.

References

[1] Statista, "Data growth worldwide 2010–2025." Accessed Nov. 2023.

[2] Baeldung, "What is content-based image retrieval?" Accessed Nov. 2023.

[3] G. Kumar, P. K. Bhatia, "A detailed review of feature extraction in image processing systems," ACCT, 2014.

[4] D. G. Lowe, "Object recognition from local scale-invariant features," ICCV, 1999.

[5] J. Hosang, R. Benenson, B. Schiele, "Learning non-maximum suppression," CVPR, 2017.

[6] M.-E. Nilsback, A. Zisserman, "Automated flower classification over a large number of classes," ICVGIP, 2008.

[7] S. Gross, "PEP 703: Making the Global Interpreter Lock optional in CPython," 2023.

[8] M. Singh et al., "A comparative study of feature extraction techniques and similarity measures for image retrieval," INCOFT, 2022.

Thank you

github.com/musawaghu/parallel-sift

NMS-SIFT code originally by Sairaj Jadhav and Dr. John Korah. Code, data, notebook and charts are in the repository.